

Introduction to Software Maintenance (SB5-MAI)

Jan Corfixen Sørensen

University of Southern Denmark

Outline

SB5-MAI: Software Maintenance

Introduction

Paradigms

Software Life-Span Models

Summary

SB5-MAI: Software Maintenance

About SB5-MAI

- ▶ **Weight:** 5 ECTS / 0.083 full-time equivalent.
- ▶ **Prerequisites:**
 - ▶ Advanced skills in object-oriented programming (equivalent to 12.5 ECTS).
 - ▶ Software Engineering (process modeling, requirements, analysis, design, architecture, corresponding to 20 ECTS).
 - ▶ Project team work.
- ▶ **Learning Outcome:** Maintenance of larger existing software projects.
- ▶ **Teaching language:** English or Danish.

SB5-MAI Exam

- ▶ Written report on campus.
The individual portfolio will constitute the basis for examination.
- ▶ The individual report is assessed using the 7-point grading scale.



SB5-MAI Portfolio



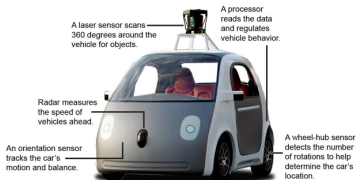
- ▶ Refactor JHotDraw features according to Clean Code.
- ▶ **Individual** mandatory portfolio examples.

Introduction

Motivation

► Internet in everything

Self driving cars



Source: Google

Raoul Raïoa / @latimesgraphics

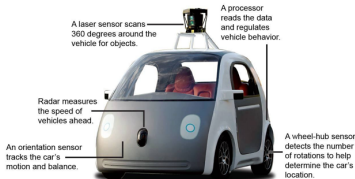
Washing Machines



Motivation

- ▶ Internet in everything
- ▶ Software is everywhere

Self driving cars



Source: Google

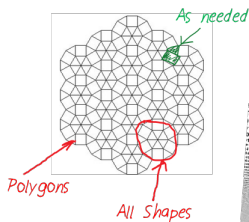
Raoul Rafoa / @latimesgraphics

Washing Machines

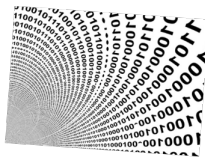


Essential Properties

Complexity



Invisibility



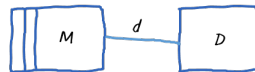
Visualization tools



Changeability



Conformity



Discontinuity



Accidental Properties

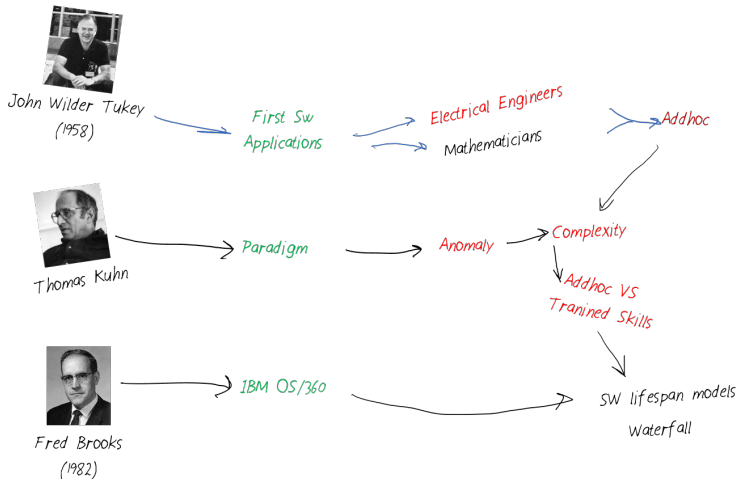
Technologies



Operating systems

Roles of Software

Software Engineering History



Paradigms

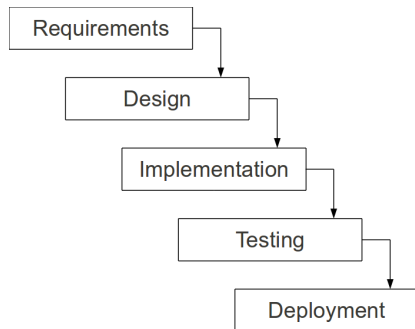
Ad hoc



- ▶ Programmers recruited from the ranks of hardware engineers and mathematicians
- ▶ Software separated from the hardware in 1950's
- ▶ Software engineering defined as a field on it's own.

Software Waterfall

- ▶ Linear process
- ▶ Used in construction and manufacturing
- ▶ Intuitively appealing metaphor



Anomaly of Requirements volatility



How the customer explained it.



How the project leader understood it.



How the analyst designed it.



How the programmer wrote it.



What the customer really wanted.

- Requirements for IT change at a rate 2 to 3 % per month (Casper-Jones).

Standish Group Anomaly

MODERN RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	29%	27%	31%	28%	29%
CHALLENGED	49%	56%	50%	55%	52%
FAILED	22%	17%	19%	17%	19%

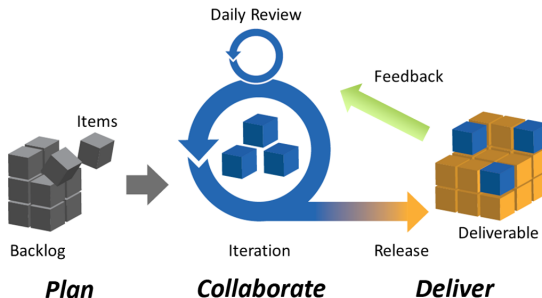
The Modern Resolution (OnTime, OnBudget, with a satisfactory result) of all software projects from FY2011-2015 within the new CHAOS database. Please note that for the rest of this report CHAOS Resolution will refer to the Modern Resolution definition not the Traditional Resolution definition.

- ▶ The CHAOS Reports published every year since 1994.
- ▶ 50,000 projects around the world.
- ▶ Based on new definition of success factors (on time, on budget with a satisfactory result).

Agile Approach

<i>Software Type</i>	<i>Monthly Rate of Requirements Change</i>
<i>Contract or outsource software</i>	<i>1.0 %</i>
<i>Information systems</i>	<i>1.5 %</i>
<i>System software</i>	<i>2.0 %</i>
<i>Military software</i>	<i>2.0 %</i>
<i>Commerstial software</i>	<i>3.5 %</i>

From „Strategies for Managing Requirements Creep“, C. Jones, 1996, IEEE Computer



Agile VS Waterfall

CHAOS RESOLUTION BY AGILE VERSUS WATERFALL

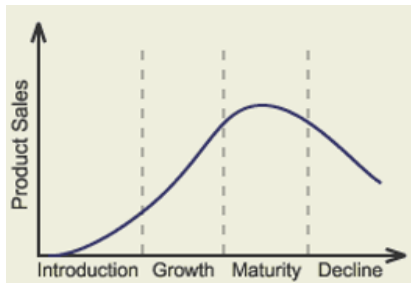
SIZE	METHOD	SUCCESSFUL	CHALLENGED	FAILED
All Size Projects	Agile	39%	52%	9%
	Waterfall	11%	60%	29%
Large Size Projects	Agile	18%	59%	23%
	Waterfall	3%	55%	42%
Medium Size Projects	Agile	27%	62%	11%
	Waterfall	7%	68%	25%
Small Size Projects	Agile	58%	38%	4%
	Waterfall	44%	45%	11%

The resolution of all software projects from FY2011-2015 within the new CHAOS database, segmented by the agile process and waterfall method. The total number of software projects is over 10,000.

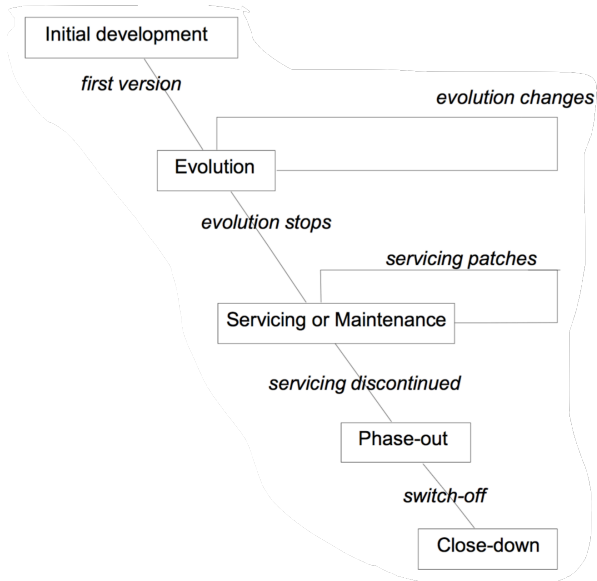
Software Life-Span Models

Life-Span Models

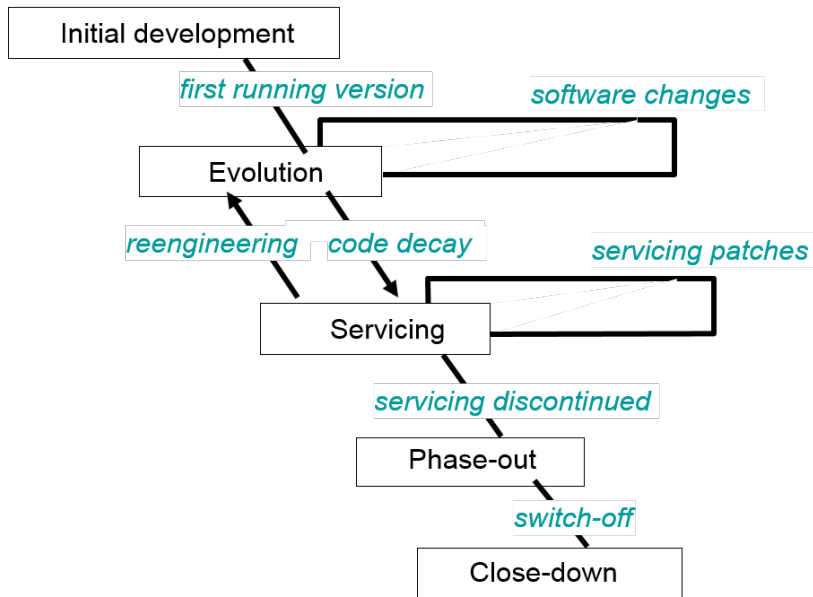
- ▶ Stages through which software goes, from conception to death.
- ▶ Stages may be very different.
- ▶ Stages are similar to the stages in the life span of other products.



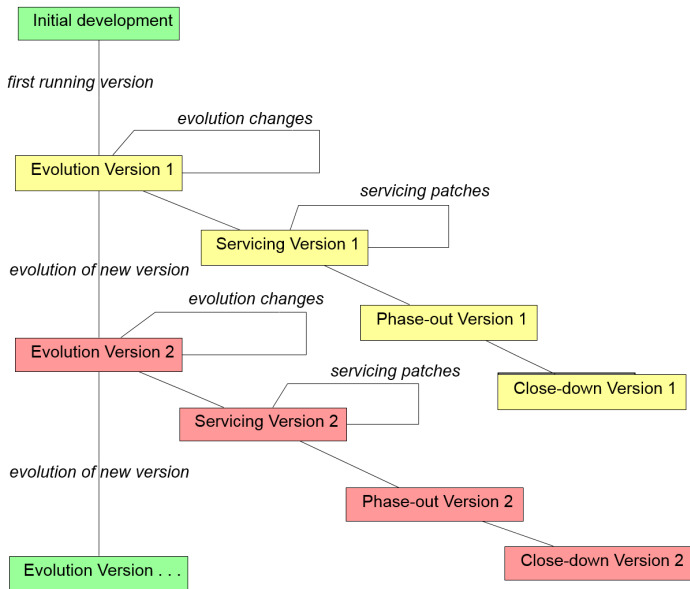
Staged Model



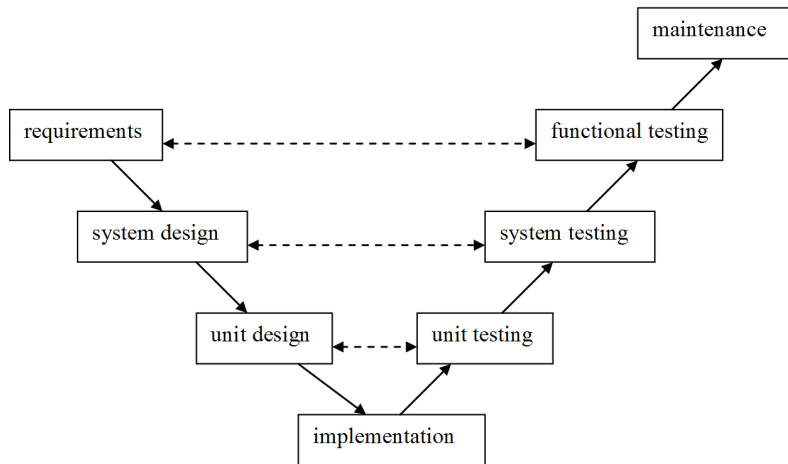
Reengineering



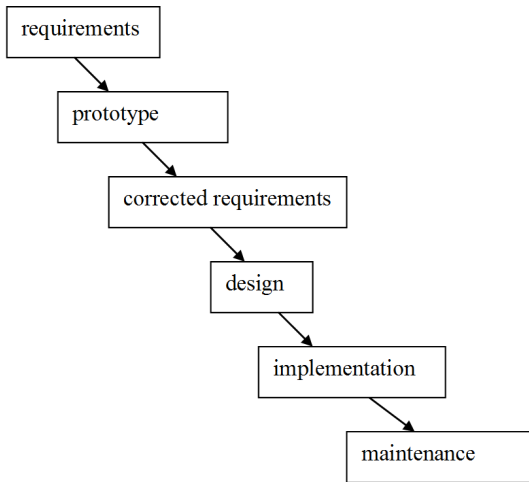
Versioned Staged Model



V-Model



Prototype Model



Summary

Software Engineering

- ▶ Software as a product since 1950s.
- ▶ Software engineers are faced with essential difficulties of complexity, invisibility, changeability, conformatity, and discontinuity.
- ▶ Three different paradigms:
 - ▶ Ad hoc
 - ▶ Waterfall
 - ▶ Iterative
- ▶ The different paradigms co-exists.

Software Life-Span Models

- ▶ Life-span models offer a simplified and comprehensive view of the entire software engineering discipline.
- ▶ Staged model
- ▶ Versioned staged model
- ▶ V-model
- ▶ Prototype